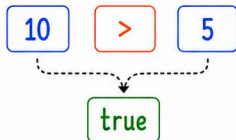




1. What is a Conditional?

A conditional helps code make a decision based on a true or false test.



2. Why Conditionals Matter

- They control program flow.
- They help make decisions.
- They validate input.
- They create different outcomes.

3. if

if

Runs code when a condition is true.

```
if (score >= 60) {  
  ...  
}
```

4. else

else

Runs when the condition is false.

```
else {  
  ...  
}
```

5. else if

else if

Checks another condition.

```
else if (score  
  >= 90) {  
  ...  
}
```

6. true

true

A condition can return true.

```
10 > 5  
// true
```

7. false

false

A condition can return false.

```
3 > 8  
// false
```





8. Condition Syntax

Use if to start a condition block.

```
if (isLoggedIn) {  
  console.log("Welcome");  
}
```



9. If / Else

Choose between two outcomes.

```
if (isActive) {  
  status = "Online";  
} else {  
  status = "Offline";  
}
```



10. else if Chain

Check multiple possible outcomes.

```
if (score >= 90) grade = "A";  
else if (score >= 75)  
  grade = "B";
```



11. Comparison in Conditions

Conditions often use
>, <, >=, <=, and ===.

```
score >= 60
```



12. Equality Check

Prefer strict equality
in conditionals.

```
role === "admin" // true
```

Lesson 4: Conditionals



13. Logical AND

Both conditions must be true.

```
isLoggedIn && isAdmin
```



14. Logical OR

At least one condition must be true.

```
isAdmin || isEditor
```



15. Logical NOT

Reverses a boolean value.

```
!isLoggedIn
```



16. Combining Conditions

Conditions can work together in one check.

```
age >= 18 && hasTicket
```





1. Nested if Statements

One condition can be inside another.

```
if (isLoggedIn) {  
  if (isAdmin) {  
    console.log("Welcome Admin");  
  }  
}
```



2. Condition Order

Code checks conditions from top to bottom.

```
if (score >= 90) ...  
else if (score >= 75) ...
```



3. Parentheses in Conditions

Use parentheses to group logic clearly.

```
(age >= 18 && hasId) || isGuardian
```



Parentheses improve clarity.



4. Strict Equality

Use === instead of == in most cases.

```
5 === "5" // false
```



Exact comparison is safer.



1. String Comparison

Conditions can compare text values.

```
username === "Ayla"
```



2. Changing a Value

Conditions can update variables.

```
let message = "";  
if (score >= 60) message = "Passed";
```



New value: Passed



3. Quick Example

Check a login result.

```
const isActive = true;  
const password = "12345";  
let access = isActive && password === "12345";
```

```
access = true
```



4. Common Mistakes

- Using = instead of ===
- Forgetting curly braces
- Wrong condition order
- Missing an else case



Small conditional mistakes can change your result.



1. Practice

Try these steps yourself:

- Create score and attendance variables.
- Use if, else if, and else.
- Check whether the student passed.
- Print the result with console.log().

```
let score = 82;  
let attendance = 90;
```



2. Challenge Answer

If the code is:

```
let score = 82;  
let result;  
if (score >= 90) result = "Excellent";  
else if (score >= 75) result = "Good";  
else result = "Needs Work";
```

- result -> "Good"



3. Mini Project

Build a simple Student Grade Checker.

```
const studentName = "Ali";  
const score = 82;  
let result = "";  
if (score >= 90) result = "Excellent";  
else if (score >= 75) result = "Good";  
else if (score >= 60) result = "Passed";  
else result = "Failed";
```

Print with console.log(studentName + " got " + result);



4. Key Takeaway

- Conditionals help programs choose actions.
- if, else if, and else create branches.
- Comparison operators return true or false.
- Logical operators combine conditions.
- Clear condition order prevents bugs.





خلاصه درس ۴

شرطها در جاوااسکریپت

Lesson 04 Summary



1 شرط چیست؟



شرط (Conditional) راهی برای تصمیم‌گیری در برنامه است. با شرطها مشخص می‌کنیم اگر یک وضعیت درست بود، یک بخش از کد اجرا شود و اگر درست نبود، مسیر دیگری اجرا شود.

در جاوااسکریپت معمولاً از if، else if و else برای این کار استفاده می‌کنیم.

2 چرا به شرطها نیاز داریم؟



تصمیم‌گیری
در برنامه



اجرای کد بر اساس
وضعیت



بررسی
چند حالت



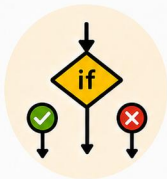
کنترل جریان
برنامه



شرطها باعث می‌شوند برنامه بر اساس داده‌ها رفتار متفاوتی داشته باشد.



3 ساختار if



دستور if زمانی اجرا می‌شود که شرط داخل پرانتز true باشد. اگر شرط درست نباشد، کد داخل آن اجرا نمی‌شود.

```
let age = 20;  
if (age >= 18) {  
  console.log("Adult");  
}
```

4 مثال سریع



```
let score = 80;  
if (score >= 50) {  
  console.log("Passed");  
}
```

اگر شرط برقرار باشد، پیام Passed چاپ می‌شود.



خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary



5 بخش else

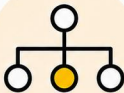
از `else` زمانی استفاده می‌کنیم که شرط `if` درست نباشد و بخواهیم مسیر دیگری اجرا شود.



```
let age = 15;  
if (age >= 18) {  
  console.log("Adult");  
} else {  
  console.log("Minor");  
}
```

6 بخش else if

اگر چند حالت مختلف داشته باشیم، از `else if` استفاده می‌کنیم تا شرط‌های بیشتری را بررسی کنیم.



```
let score = 75;  
if (score >= 90) {  
  console.log("A");  
} else if (score >= 70) {  
  console.log("B");  
} else {  
  console.log("C");  
}
```



برای بررسی چند حالت، `else if` بسیار کاربردی است.



JS

خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary

{ }

7 عملگرهای مقایسه‌ای

>

بزرگ‌تر از

<

کوچک‌تر از

>=

بزرگ‌تر یا مساوی

<=

کوچک‌تر یا مساوی

===

مساوی دقیق

!==

نامساوی دقیق

8 نتیجه شرط



10 > 5



true



7 < 4



false



5 === 5



true



3 === "3"



false

شرطها در نهایت مقدار true یا false تولید می‌کنند.

خروجی هر شرط
یک مقدار بولی است.



خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary



عملگرهای منطقی

9

&&

و: هر دو شرط
باید درست باشند.

||

یا: حداقل یکی از
شرطها باید درست باشد.

!

نقیض: نتیجه شرط را
برعکس می‌کند.

مثال‌های ساده

10

1 `age >= 18 && hasId`

اگر سن ۱۸ یا بیشتر است و کارت شناسایی دارد

2 `isAdmin || isEditor`

اگر ادمین است یا ادیتور است

3 `!isLoggedIn`

اگر وارد نشده است



عملگرهای منطقی برای ترکیب شرطها استفاده می‌شوند.





خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary



11 ترکیب شرطها



می‌توانیم شرط‌های مقایسه‌ای و منطقی را با هم ترکیب کنیم تا تصمیم دقیق‌تری بگیریم.

```
(score >= 50 && attendance >= 80) || projectDone
```

یعنی اگر نمره حداقل ۵۰ و حضور حداقل ۸۰ باشد، یا پروژه انجام شده باشد، شرط برقرار است.

12 ترتیب اجرا



1 پرانتزها ()

2 مقایسه‌ها

3 عملگرهای منطقی



برای جلوگیری از اشتباه، در عبارت‌های پیچیده از پرانتز استفاده کن.



JS

خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary

{ }

13 شرط تو در تو

```
if (isLoggedIn) {  
  if (isAdmin) {  
    console.log("Admin Panel");  
  }  
}
```

گاهی یک شرط را داخل شرط دیگری می‌نویسیم. به این حالت شرط تو در تو یا nested if می‌گویند.

14 مثال کاربردی

```
let isLoggedIn = true;  
let isAdmin = false;  
if (isLoggedIn) {  
  if (isAdmin) {  
    console.log("Access Granted");  
  } else {  
    console.log("User Access");  
  }  
}
```

در این مثال، اگر کاربر وارد شده باشد، اگر ادمین باشد دسترسی کامل دارد، در غیر این صورت دسترسی محدود خواهد داشت.



شرطهای تو در تو برای بررسی مرحله‌ای وضعیت‌ها کاربرد دارند.





خلاصه درس ۴

شرطها در جاوااسکریپت

Lesson 04 Summary



15 اپراتور سه‌تایی ؟:



اپراتور سه‌تایی شکل کوتاه‌تری از if / else است.

```
let message = age >= 18 ? "Adult" : "Minor";
```

اگر شرط درست باشد مقدار اول، و اگر نادرست باشد مقدار دوم انتخاب می‌شود.

16 نکات مهم



از === به جای == استفاده کن ✓

نام متغیرها را واضح انتخاب کن ✓

در شرط‌های پیچیده از پرانتز کمک بگیر ✓

از تو در تو شدن زیاد شرط‌ها پرهیز کن ✓



خوانا بودن شرط‌ها به فهم بهتر کد کمک می‌کند.





خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary



17 یک مثال کامل

```
let score = 75;  
let attendance = 90;  
const passed = score >= 50 && attendance >= 80;  
  
if (passed) {  
  console.log("Passed");  
} else {  
  console.log("Failed");  
}
```

18 نتیجه

passed



true

پیام خروجی



Passed

چون هم نمره و هم حضور مناسب هستند، نتیجه شرط true می شود.





خلاصه درس ۴

شرطها در جاوا اسکریپت

Lesson 04 Summary



تمرین پیشنهادی

19

- دو متغیر score و attendance تعریف کن.
- با if بررسی کن آیا score حداقل ۵۰ است یا نه.
- با && بررسی کن آیا attendance حداقل ۸۰ است یا نه.
- اگر هر دو درست بودند، پیام Passed را چاپ کن.
- در غیر این صورت پیام Failed را چاپ کن.



تا اینجا باید چه چیزهایی بلد باشیم؟

20

- ✓ if
- ✓ else
- ✓ else if
- ✓ عملگرهای مقایسه‌ای
- ✓ عملگرهای منطقی
- ✓ شرط تو در تو
- ✓ اپراتور سه تایی



اگر این بخش‌ها را خوب یاد بگیری، نوشتن تصمیم‌ها در کد برایت ساده‌تر می‌شود.



1

تمرین پیشنهادی



- دو متغیر عددی به نام‌های score و age تعریف کن.
- با if بررسی کن آیا score بزرگ‌تر یا مساوی ۱۰ هست.
- با if / else مشخص کن کاربر adult است یا minor.
- یک شرط ترکیبی با && بساز: score >= 10 && age >= 18
- نتیجه‌ها را با console.log() چاپ کن.

2

Challenge جواب



```
let score = 14;
let age = 19;

if (score >= 10) {
  console.log("Passed");
} else {
  console.log("Failed");
}

if (age >= 18) {
  console.log("Adult");
} else {
  console.log("Minor");
}

console.log(score >= 10 && age >= 18); // true
```

- چون score برابر ۱۴ است، نتیجه Passed می‌شود.
- چون age برابر ۱۹ است، نتیجه Adult می‌شود.
- شرط ترکیبی هم true برمی‌گرداند.



3 مینی پروژه

یک Simple Access Checker با استفاده از شرطها بساز.

```
const username = "Ayla";
const isLoggedIn = true;
const isAdmin = false;

if (isLoggedIn && isAdmin) {
  console.log("Welcome admin");
} else if (isLoggedIn) {
  console.log("Welcome user");
} else {
  console.log("Please log in");
}
```

- چون کاربر وارد شده اما ادمین نیست،
- پیام Welcome user نمایش داده می‌شود.
- اگر وارد نشده بود، پیام
- Please log in نمایش داده می‌شد.



4 جمع‌بندی نهایی

- if برای اجرای کد در صورت درست بودن شرط استفاده می‌شود.
- else مسیر جایگزین را اجرا می‌کند.
- else if چندین حالت مختلف را بررسی می‌کند.
- عملگرهای مقایسه‌ای و منطقی به ساخت شرط کمک می‌کنند.
- شرطها باعث تصمیم‌گیری در برنامه می‌شوند.

حالا آماده‌ای با شرطها منطق برنامه‌هایت را در جاوا اسکریپت بسازی.



خروجی تمرین جاوااسکریپت

JavaScript Practice Output

The screenshot shows the Visual Studio Code interface. On the left is the Explorer sidebar with a file tree for 'LEARNING-JAVASCRIPT'. The tree includes folders 'docs', 'lesson-01', 'lesson-02', 'lesson-03', and 'lesson-04'. Inside 'lesson-04' is a folder 'practice-01' containing 'index.html', 'README.md', and 'script.js'. The 'script.js' file is selected. The main editor area shows the content of 'script.js' with the following code:

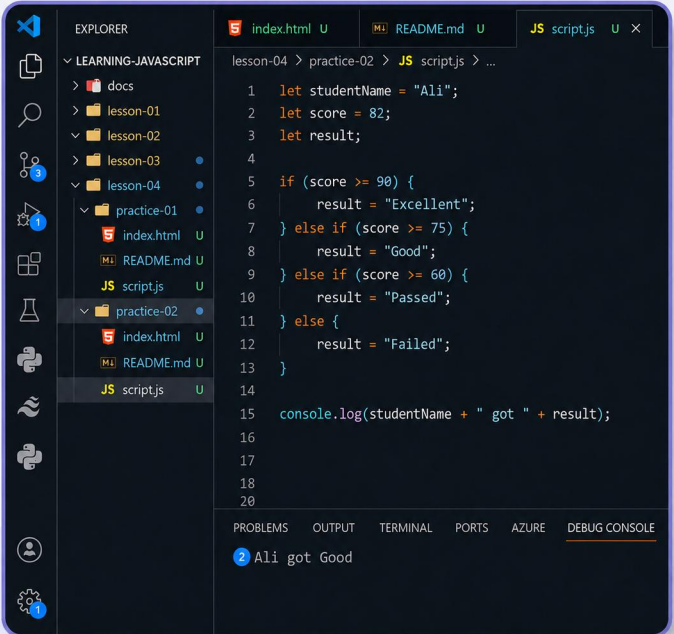
```
1    
2  let username = "admin";  
3  let password = "12345";  
4  let isActive = true;  
5  
6  if (  
7      username === "admin" &&  
8      password === "12345" &&  
9      isActive === true  
10 ) {  
11     console.log("Login successful");  
12 } else {  
14     console.log("Login failed");  
15 }
```

At the bottom of the editor is the 'DEBUG CONSOLE' tab, which shows the output:

```
2 Login successful
```

خروجی تمرین جاوااسکریپت

JavaScript Practice Output



The screenshot shows the Visual Studio Code interface. On the left is the Explorer sidebar with a file tree. The main editor area displays the code in `script.js`. The bottom status bar shows the output: `2 Ali got Good`.

EXPLORER

- LEARNING-JAVASCRIPT
 - docs
 - lesson-01
 - lesson-02
 - lesson-03
 - lesson-04
 - practice-01
 - index.html
 - README.md
 - script.js
 - practice-02
 - index.html
 - README.md
 - script.js

index.html **README.md** **JS script.js**

lesson-04 > practice-02 > JS script.js > ...

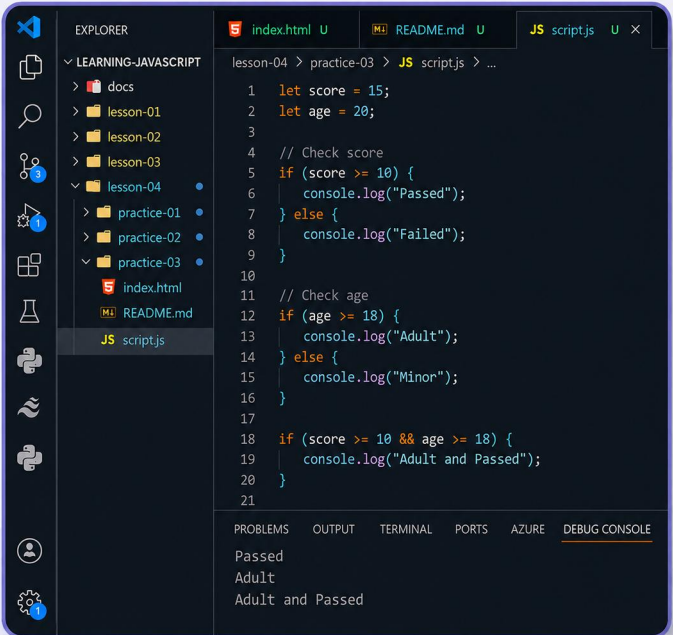
```
1 let studentName = "Ali";
2 let score = 82;
3 let result;
4
5 if (score >= 90) {
6     result = "Excellent";
7 } else if (score >= 75) {
8     result = "Good";
9 } else if (score >= 60) {
10    result = "Passed";
11 } else {
12    result = "Failed";
13 }
14
15 console.log(studentName + " got " + result);
16
17
18
19
20
```

PROBLEMS **OUTPUT** **TERMINAL** **PORTS** **AZURE** **DEBUG CONSOLE**

2 Ali got Good

خروجی تمرین جاوا اسکریپت

JavaScript Practice Output



The image shows a VS Code editor window with a dark theme. The Explorer sidebar on the left shows a project structure for 'LEARNING-JAVASCRIPT' with folders 'docs', 'lesson-01', 'lesson-02', 'lesson-03', and 'lesson-04'. Inside 'lesson-04' are folders 'practice-01', 'practice-02', and 'practice-03'. The 'practice-03' folder is selected, showing 'index.html' and 'script.js'. The 'script.js' file is open in the editor, displaying JavaScript code that checks a score and age. The output console at the bottom shows the results of the code execution.

EXPLORER

LEARNING-JAVASCRIPT

- > docs
- > lesson-01
- > lesson-02
- > lesson-03
- > lesson-04
 - > practice-01
 - > practice-02
 - > practice-03
 - index.html
 - README.md
 - script.js

index.html U README.md U JS script.js U X

lesson-04 > practice-03 > JS script.js > ...

```
1 let score = 15;
2 let age = 20;
3
4 // Check score
5 if (score >= 10) {
6     console.log("Passed");
7 } else {
8     console.log("Failed");
9 }
10
11 // Check age
12 if (age >= 18) {
13     console.log("Adult");
14 } else {
15     console.log("Minor");
16 }
17
18 if (score >= 10 && age >= 18) {
19     console.log("Adult and Passed");
20 }
21
```

PROBLEMS OUTPUT TERMINAL PORTS AZURE DEBUG CONSOLE

Passed
Adult
Adult and Passed

خروجی تمرین جاوااسکریپت

JavaScript Practice Output

The screenshot shows the Visual Studio Code interface. On the left is the Explorer sidebar with a file tree for 'LEARNING-JAVASCRIPT'. The tree includes folders for 'docs', 'lesson-01', 'lesson-02', 'lesson-03', and 'lesson-04'. Under 'lesson-04', there are folders for 'practice-01', 'practice-02', 'practice-03', and 'practice-04'. Inside 'practice-04', the files 'index.html' and 'README.md' are listed, and 'JS script.js' is selected. The main editor area displays the content of 'script.js', which contains a JavaScript snippet that checks login status and admin privileges. The bottom panel shows the 'DEBUG CONSOLE' tab with the output 'Welcome User Ayla'.

```
1  const username = "Ayla";
2  const isLoggedIn = true;
3  const isAdmin = false;
4
5  if (isLoggedIn && isAdmin) {
6      console.log("Welcome Admin " + username);
7  } else if (isLoggedIn) {
8      console.log("Welcome User " + username);
9  } else {
10     console.log("Please Login");
11 }
12
13
14
15
16
17
18
```

PROBLEMS OUTPUT TERMINAL PORTS AZURE DEBUG CONSOLE

Welcome User Ayla